

Q. Explain NLP issues. Also explain Expert System architecture.

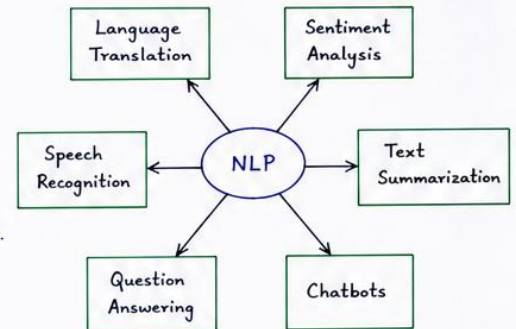
Ans. 1 NATURAL LANGUAGE PROCESSING (NLP) - ISSUES

Natural Language Processing (NLP) is a field of AI that enables computers to understand, interpret and generate human language. Despite significant progress, NLP faces several challenges.

MAJOR ISSUES IN NLP :

1. Ambiguity : Words and sentences can have multiple meanings.  
Example : "bank" → can mean financial bank or river bank.
2. Context Understanding : The meaning of a word depends on the context in which it is used.  
Example : "I saw him on the bank" (needs context to decide meaning).
3. Language Complexity : Human language has complex grammar, idioms, sarcasm, slang etc.  
Example : "That's cool!" (may mean appreciation or indifference).
4. Data Sparsity : Many words/phrases occur rarely, lack of sufficient training data affects accuracy.
5. Variations in Language : Different languages, accents, dialects and informal expressions make processing difficult.
6. World Knowledge : Machines lack real world knowledge required to understand implicit meaning.
7. Computational Cost : NLP tasks like parsing, translation and summarization require high computational resources.

NLP TASKS (Examples)



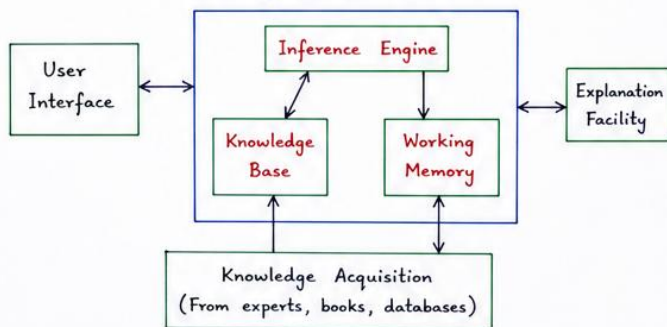
EXAMPLE :

- In Machine Translation, the word "right" may mean "correct" or "right side".
- In Sentiment Analysis, the sentence "This movie was sick!" may be positive (very good) in informal usage.

Ans. 2 EXPERT SYSTEM ARCHITECTURE

An Expert System is an AI program that simulates the decision-making ability of a human expert in a specific domain. It uses knowledge and reasoning to solve complex problems.

ARCHITECTURE / COMPONENTS :



EXAMPLE :

- In a Medical Expert System, symptoms entered by the user are matched with rules in the knowledge base.
- The inference engine concludes the possible disease and suggests treatment.

1. User Interface : Allows user to input facts, ask questions and get solutions.
2. Knowledge Base : Contains domain knowledge, facts, rules and heuristics.
3. Inference Engine : Applies rules on facts in working memory to derive new facts or conclusions.
4. Working Memory : Stores facts about the current problem.
5. Explanation Facility : Explains how the system reached a particular conclusion.
6. Knowledge Acquisition Module : Helps in gathering and updating knowledge from experts or other sources.

## Q. Explain Supervised and Unsupervised learning. Also explain Decision Trees.

### Ans. 1. SUPERVISED AND UNSUPERVISED LEARNING

Machine Learning techniques are broadly divided into two types : Supervised Learning and Unsupervised Learning.

#### \* Supervised Learning :

- Definition : In supervised learning, the algorithm is trained using labelled data. Each training example has an input and a corresponding output.
- Goal : To learn a mapping from inputs to outputs and predict the output for new unseen data.
- How it works : The model learns a function  $f(x)$  from training set  $\{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$  and predicts  $y$  for a new  $x$ .
- Types : Classification and Regression.
- Example :
  - Email spam detection (Spam / Not spam)
  - Predicting house price from features.

#### \* Unsupervised Learning :

- Definition : In unsupervised learning, the algorithm is given unlabelled data. It finds patterns or structure in the data without any target output.
- Goal : To discover hidden patterns, groups or relationships in the data.
- How it works : The model explores the data and clusters or reduces dimensions based on similarity.
- Types : Clustering and Dimensionality Reduction.
- Example :
  - Customer segmentation using K-means clustering.
  - Grouping similar documents.

DIFFERENCE BETWEEN SUPERVISED AND UNSUPERVISED LEARNING

| Feature    | Supervised Learning                    | Unsupervised Learning                 |
|------------|----------------------------------------|---------------------------------------|
| Data       | Labelled data                          | Unlabelled data                       |
| Goal       | Predict output                         | Discover patterns                     |
| Output     | Specific (Class/Value)                 | General (Clusters/Groups)             |
| Examples   | Classification, Regression             | Clustering, Association               |
| Algorithms | Linear Regression, SVM, Decision Trees | K-means, Hierarchical Clustering, PCA |

### Ans. 2. DECISION TREES

A Decision Tree is a flowchart-like structure in which internal nodes represent tests on attributes, branches represent outcomes of the tests and leaf nodes represent class labels or decisions.

#### \* Construction of Decision Tree :

1. Start with the entire training set at the root.
2. Select the best attribute using a measure such as Information Gain or Gini Index.
3. Split the data into subsets based on the attribute values.
4. Repeat the process recursively for each subset.
5. Stop when all records in a node belong to the same class or no attributes are left.

#### \* Advantages :

- Easy to understand and interpret.
- Requires little data preparation.
- Handles both numerical and categorical data.
- Can model non-linear relationships.

#### \* Limitations :

- Can be unstable (small change in data may change the tree).
- Prone to overfitting.

#### \* Example : Consider the following training data to decide whether a person will play Tennis.

| Outlook  | Temperature | Humidity | Wind   | Play Tennis |
|----------|-------------|----------|--------|-------------|
| Sunny    | Hot         | High     | Weak   | No          |
| Sunny    | Hot         | High     | Strong | No          |
| Overcast | Hot         | High     | Weak   | Yes         |
| Rain     | Mild        | High     | Weak   | Yes         |
| Rain     | Cool        | Normal   | Weak   | Yes         |
| Rain     | Cool        | Normal   | Strong | No          |
| Overcast | Cool        | Normal   | Strong | Yes         |
| Sunny    | Mild        | High     | Weak   | No          |
| Sunny    | Cool        | Normal   | Weak   | Yes         |
| Rain     | Mild        | Normal   | Weak   | Yes         |

Using Information Gain, the decision tree constructed is :



This tree can be used to predict "Play Tennis" for any new instance.



Q. Explain Hill Climbing with an example.  
Explain Heuristic function.

1. HILL CLIMBING

**Definition:** Hill Climbing is a local search algorithm that starts with an initial state and moves to a neighboring state that has a better value according to an evaluation function. It continues until no better neighbor is found.

Explanation / Working :

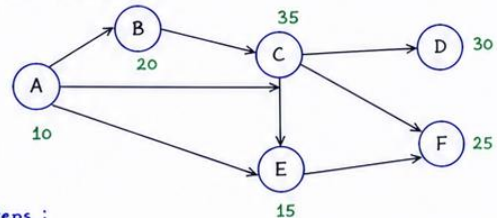
1. Start from an initial state.
2. Evaluate all the neighboring states using an evaluation function.
3. Move to the neighbor with the best (highest) value.
4. Repeat the process.
5. Stop when no neighbor has a better value.

Characteristics :

- Uses only local information.
- Greedy strategy.
- May get stuck at local maxima, plateaus or ridges.

Example :

Consider the following state space. The value in each state represents the evaluation (to be maximized). Start at state A.



Steps :

- Start at A (10). Neighbors  $\rightarrow \{B(20), C(35), E(15)\}$   
Best is C (35). Move to C.
- From C, neighbors  $\rightarrow \{B(20), D(30), E(15), F(25)\}$   
Best is D (30). Move to D.
- From D, neighbor  $\rightarrow \{C(35)\}$  which is not better.  
So, stop.

Therefore, Hill Climbing terminates at D with value 30.  
 Note: Global maximum is 35 at C, but algorithm got stuck at local maximum (D = 30).

2. HEURISTIC FUNCTION

**Definition:** A Heuristic function is a function  $h(n)$  that estimates the cost from the current node  $n$  to the goal state. It is used to guide the search towards the goal and to reduce the search space.

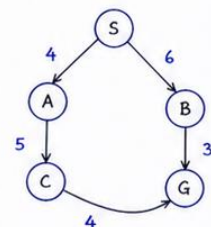
Explanation / Key Points :

1. Heuristic gives an estimate, not exact cost.
2. It helps in informed search algorithms like Greedy Best-First Search and A\* Search.
3. A good heuristic is:
  - o Admissible : Never overestimates the true cost to the goal. ( $h(n) \leq h^*(n)$ )
  - o Consistent : For every node  $n$  and its successor  $n'$ ,  $h(n) \leq c(n, n') + h(n')$ .
4. Better heuristic  $\rightarrow$  fewer nodes expanded  $\rightarrow$  efficient search.
5. If  $h(n) = 0$  for all  $n$ , the search becomes equivalent to Uniform Cost Search.

Example :

Consider a path finding problem from Start (S) to Goal (G). The straight-line distance (in km) to the goal is used as heuristic  $h(n)$ .

| Node | $h(n)$ (Estimated distance to Goal) |
|------|-------------------------------------|
| S    | 14                                  |
| A    | 10                                  |
| B    | 7                                   |
| C    | 4                                   |
| G    | 0                                   |



Using  $h(n)$  the search will prefer nodes that are closer to goal. For example, from S it will choose B ( $h=7$ ) over A ( $h=10$ ). Finally it reaches G where  $h(G)=0$ .

Hence, heuristic function provides an intelligent estimate to guide the search towards the goal efficiently.

## Q. Explain Neural Networks. Explain SVM.

### 1. NEURAL NETWORKS

**Definition :** A Neural Network (NN) is a computing model inspired by the human brain. It consists of interconnected neurons (nodes) organized in layers. NN learns from examples and is used for pattern recognition, classification and prediction.

#### Architecture :

A typical feed-forward neural network has three types of layers.

1. Input Layer - accepts input features.
2. Hidden Layer(s) - performs computation using weights and activation function.
3. Output Layer - produces final output.

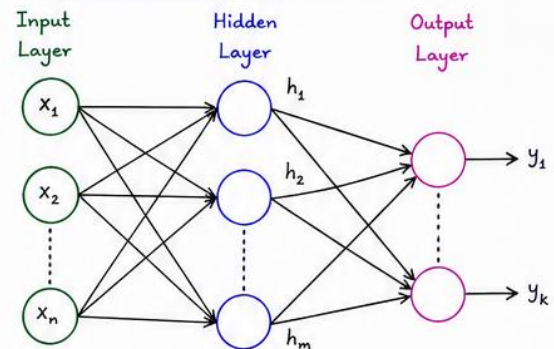
#### Working (Forward Propagation) :

1. Each input is multiplied by a weight.
2. The weighted inputs are summed and bias is added.
3. Activation function is applied to produce output.
4. The process continues layer by layer till output layer.

#### Common Activation Functions :

Sigmoid  $f(z) = \frac{1}{1+e^{-z}}$ , Tanh  $f(z) = \tanh(z)$ , ReLU  $f(z) = \max(0, z)$

#### Structure of a Neural Network



$$\text{Net input to hidden neuron } j : z_j = \sum_{i=1}^n x_i w_{ij} + b_j$$

$$\text{Output of hidden neuron : } h_j = f(z_j) \text{ (activation function)}$$

#### Example :

Consider XOR problem.

| Input 1 | Input 2 | Desired Output |
|---------|---------|----------------|
| 0       | 0       | 0              |
| 0       | 1       | 1              |
| 1       | 0       | 1              |
| 1       | 1       | 0              |

A neural network with one hidden layer can learn this non-linear relationship which is not possible using a single linear model.

### 2. SUPPORT VECTOR MACHINE (SVM)

**Definition :** Support Vector Machine (SVM) is a supervised learning algorithm used for classification and regression. It works by finding the optimal hyperplane that best separates the classes with the maximum margin.

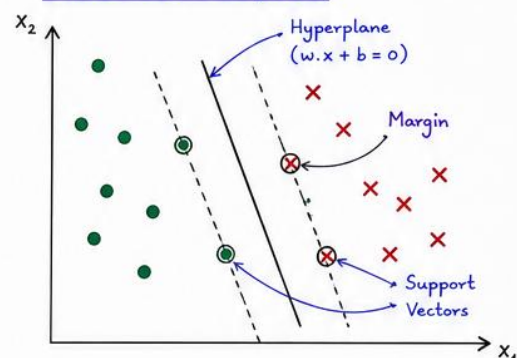
#### Key Points :

1. Finds the best decision boundary (hyperplane).
2. Maximizes the margin between classes.
3. Only a subset of training points called Support Vectors lie on the margin.
4. Works with linear and non-linear data (using kernels).
5. Used in classification, regression, and anomaly detection.

#### Types of SVM :

1. Linear SVM - for linearly separable data.
2. Kernel SVM - for non-linearly separable data using kernel trick.

#### SVM - Linear Classification



#### Example :

Given two classes in 2D space. SVM finds the line that separates them with maximum margin. The points lying on the margin are support vectors.

In Kernel SVM, data is first mapped to a higher dimensional space using a kernel function (e.g., RBF, Polynomial) and then a hyperplane is found.



- Q.1) Explain BFS, DFS (Example + Algorithm). 2) Explain A\* Search (with example).  
3) Explain Greedy Search.

### 1) BFS (Breadth First Search)

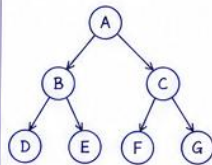
Definition: BFS explores the graph level by level.

It expands all the nodes at the present depth before moving to the next depth.

Algorithm (BFS):

1. Start from the initial node.
2. Put the start node into a queue.
3. Remove the front node from queue and visit it.
4. Insert all its unvisited adjacent nodes into the queue.
5. Repeat steps 3-4 until the goal is found or queue becomes empty.

Example for BFS:



Starting node = A, Goal = G

BFS traversal order:

A B C D E F G  
(Level order)

Properties:

- Complete.
- Optimal (for unit cost).
- Space complexity is high.

### 2) DFS (Depth First Search)

Definition: DFS explores as far as possible along each branch before backtracking.

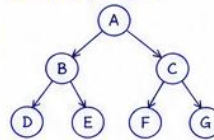
Algorithm (DFS):

1. Start from the initial node.
2. Visit the node and mark it.
3. Recur for an unvisited adjacent node.
4. If no adjacent node is left, backtrack.
5. Repeat until goal is found or all nodes are visited.

Properties:

- Not complete in infinite depth.
- Not optimal.
- Space complexity is less.

Example for DFS:



Starting node = A, Goal = G

One possible DFS order:

A B D E C F G  
(Depth wise)

### 3) A\* SEARCH

Definition: A\* (A-star) search is an informed search algorithm that uses both the actual cost from start to a node and heuristic estimate from the node to the goal.

Evaluation Function:  $f(n) = g(n) + h(n)$

where,  $g(n)$  = cost from start to n

$h(n)$  = heuristic estimate from n to goal

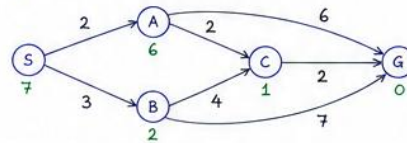
$f(n)$  = estimated total cost through n

Algorithm (A\* Search):

1. Put the start node in OPEN list with  $f(n) = g(n) + h(n)$ .
2. Choose the node with smallest  $f(n)$  from OPEN.
3. Move it to CLOSED list.
4. Expand it and generate successors.
5. For each successor, compute  $f(n)$ .  
Add to OPEN list if not present or if a better path is found.
6. Repeat until goal node is selected.
7. Reconstruct path using parent pointers.

Example for A\* Search:

Heuristic values ( $h$ ) are shown in green.  
Edge costs are shown in black.



| Step | Node Selected | $g(n)$ | $h(n)$ | $f(n)=g+h$ | OPEN List ( $f$ ) |
|------|---------------|--------|--------|------------|-------------------|
| 0    | S             | 0      | 7      | 7          | A(8), B(5)        |
| 1    | B             | 3      | 2      | 5          | A(8), C(8), G(10) |
| 2    | A             | 2      | 6      | 8          | C(4), G(8)        |
| 3    | C             | 6      | 1      | 7          | G(8)              |
| 4    | G             | 8      | 0      | 8          | Goal reached      |

Optimal path found:  $S \rightarrow B \rightarrow C \rightarrow G$

Total cost = 8

### 4) GREEDY SEARCH (Best-First Search)

Definition: Greedy search is an informed search algorithm that always expands the node that is closest to the goal according to a heuristic function. It uses only heuristic value.

Evaluation Function:  $f(n) = h(n)$

Algorithm (Greedy Search):

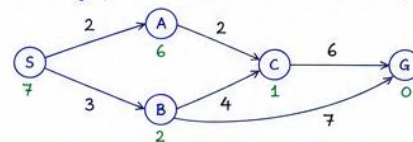
1. Put the start node in OPEN list.
2. Select the node with smallest  $h(n)$ .
3. Move it to CLOSED list.
4. Expand it and add its successors to OPEN.
5. Repeat until goal is found.

Properties:

- Fast (in many cases).
- Not optimal (may not give least cost path).
- Not complete in infinite search space.

Example for Greedy Search:

(Same graph and heuristic values as above)



| Step | Node Selected | $h(n)$ | OPEN List (by $h$ ) |
|------|---------------|--------|---------------------|
| 0    | S             | 7      | A(6), B(2)          |
| 1    | B             | 2      | C(1), A(6), G(0)    |
| 2    | G             | 0      | Goal reached        |

Path found by Greedy:  $S \rightarrow B \rightarrow G$

Total cost = 10 (Not optimal, since optimal cost is 8)

Summary:

- BFS is suitable for shortest path in unweighted graphs.
- DFS uses less memory but may not find shortest path.
- A\* search uses  $g(n) + h(n)$  and guarantees optimal path (with admissible heuristic).
- Greedy search is fast but not optimal as it uses only  $h(n)$ .

## 1. EXPLAIN MINIMAX ALGORITHM WITH AN EXAMPLE.

### Definition :

Minimax is a game playing algorithm used in two player, zero-sum games. One player tries to maximize the score (MAX) while the other tries to minimize the score (MIN). The algorithm explores the game tree and backtracks the values to choose the optimal move.

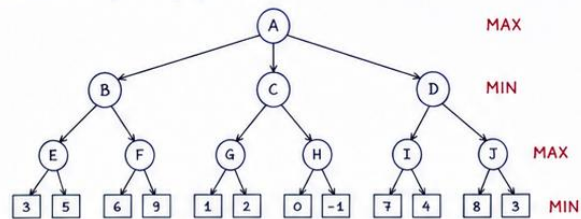
### Algorithm (Minimax) :

1. Generate the game tree from the current state.
2. Assign utility values to all terminal (leaf) nodes.
3. Propagate the values upward :
  - MAX level  $\rightarrow$  take maximum of children.
  - MIN level  $\rightarrow$  take minimum of children.
4. The value at the root node gives the best achievable value for MAX.
5. Choose the move leading to that value.

Therefore, the minimax value of the root (A) is 7.  
So, MAX should choose move leading to node D.

### Example :

Consider the following game tree.



Step 1: Evaluate leaf nodes.

Step 2: MAX level (E, F, G, H, I, J)

$$E = \max(3, 5) = 5, \quad F = \max(6, 9) = 9, \quad G = \max(1, 2) = 2, \\ H = \max(0, -1) = 0, \quad I = \max(7, 4) = 7, \quad J = \max(8, 3) = 8$$

Step 3: MIN level (B, C, D)

$$B = \min(5, 9) = 5, \quad C = \min(2, 0) = 0, \quad D = \min(7, 8) = 7$$

Step 4: Root (A) MAX level

$$A = \max(5, 0, 7) = 7$$

## 2. EXPLAIN ALPHA-BETA PRUNING WITH AN EXAMPLE.

### Definition :

Alpha-Beta pruning is an optimization of minimax algorithm. It eliminates the branches that cannot affect the final decision. It reduces the number of nodes evaluated, thus saving time and space.

### Key Ideas :

- Alpha ( $\alpha$ )  $\rightarrow$  best (highest) value found so far for MAX.
- Beta ( $\beta$ )  $\rightarrow$  best (lowest) value found so far for MIN.
- If  $\alpha \geq \beta$ , remaining children need not be explored (pruned).

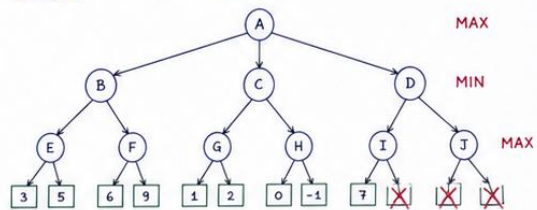
### Algorithm (Alpha-Beta Pruning) :

1. Start the search with  $\alpha = -\infty$ ,  $\beta = +\infty$ .
2. Traverse the game tree depth-first (left to right).
3. At MAX node: update  $\alpha = \max(\alpha, \text{value})$ .  
At MIN node: update  $\beta = \min(\beta, \text{value})$ .
4. If  $\alpha \geq \beta$  at any node, prune the remaining children.
5. Continue until the value of the root is determined.

### Result :

- Pruned nodes: H and its children (0, -1) and J's right child (3).
- Minimax value at root A = 7
- Best move for MAX  $\rightarrow$  choose D.

Example : Consider the same tree.



Step-by-step evaluation (left to right) :

| Node                                                                  | Type | Value                  | $\alpha$  | $\beta$                 | Action / Note     |
|-----------------------------------------------------------------------|------|------------------------|-----------|-------------------------|-------------------|
| A                                                                     | MAX  | -                      | $-\infty$ | $+\infty$               | Start             |
| B                                                                     | MIN  | -                      | $-\infty$ | $+\infty$               |                   |
| E                                                                     | MAX  | $\max(3, 5) = 5$       | $-\infty$ | 5                       | return 5 to B     |
| F                                                                     | MAX  | $\max(6, 9) = 9$       | $-\infty$ | 5 $\rightarrow$ 9       | return 9 to B     |
| B                                                                     | MIN  | $\min(5, 9) = 5$       | $-\infty$ | $+\infty \rightarrow 5$ | return 5 to A     |
| A                                                                     | MAX  | $\max(-\infty, 5) = 5$ | 5         | $+\infty$               | $\alpha$ at A = 5 |
| C                                                                     | MIN  | -                      | 5         | $+\infty$               |                   |
| G                                                                     | MAX  | $\max(1, 2) = 2$       | 5         | $+\infty$               | $\beta$ at C = 2  |
| Since $\beta(2) \leq \alpha(5) \rightarrow$ prune H and its children. |      |                        |           |                         |                   |
| D                                                                     | MIN  | -                      | 5         | $+\infty$               |                   |
| I                                                                     | MAX  | $\max(7, 4) = 7$       | 5         | 7                       | $\beta$ at D = 7  |
| Since $\beta(7) \leq \alpha(5)$ is false, but next child J:           |      |                        |           |                         |                   |

After evaluating I,  $\alpha(5) \geq \beta(7)$  is true, since  $\alpha(5) \geq \beta(7)$  is false. Actually pruning occurs because after I,  $\alpha(5) \geq \beta(7)$  is false. However at J, during exploration first leaf 8 gives  $\beta = 8$  (but  $\beta$  already 7 from I).

So when visiting J,  $\alpha(5) \geq \beta(7)$  is true  $\rightarrow$  prune remaining child (3).



## Q. Inference / Theorem Proving & Bayesian Networks in reference to Knowledge & Reasoning in AI with an example.

### 1. INFERENCE / THEOREM PROVING

#### Definition :

Inference is the process of deriving new sentences (conclusions) from a set of given sentences (knowledge base) using a formal system of logic. Theorem proving checks whether a given sentence (query) can be logically inferred from the knowledge base.

#### Key Points :

- Knowledge is represented using sentences in logic.
- A query is entailed by KB (written as  $KB \models \alpha$ ) if  $\alpha$  is true in all models of KB.
- Methods : Forward chaining, Backward chaining, Resolution.

#### Example (Using Resolution) :

KB :

1.  $P \rightarrow Q$
2.  $Q \rightarrow R$
3.  $P$

We will prove R using resolution.

Query: R

Convert KB sentences to CNF (clause form):

1.  $\neg P \vee Q$
2.  $\neg Q \vee R$
3.  $P$

Query:  $R \Rightarrow \neg R$  (negate the query)

So, clause to prove:  $\neg R$

#### Resolution Steps :

| Step | Clauses used              | Resolvent | Justification |
|------|---------------------------|-----------|---------------|
| 1.   | $(\neg P \vee Q)$ , $(P)$ | Q         | Resolve on P  |
| 2.   | $(\neg Q \vee R)$ , $(Q)$ | R         | Resolve on Q  |
| 3.   | $(R)$ , $(\neg R)$        | $\square$ | Resolve on R  |

Since we derived empty clause ( $\square$ ),  $KB \models R$

Hence, R is inferred (proved).

#### Inference Methods :

- Forward Chaining (Data-driven)  
Start from known facts and derive new facts.
- Backward Chaining (Goal-driven)  
Start from goal and work backward to known facts.
- Resolution  
Proves by refutation (derive contradiction).

#### Conclusion :

Inference helps to derive new knowledge from existing knowledge. Theorem proving techniques like resolution guarantee correctness of the derived conclusions.

### 2. BAYESIAN NETWORKS

#### Definition :

A Bayesian Network (BN) is a graphical model that represents a set of random variables and their conditional dependencies using a Directed Acyclic Graph (DAG). It allows us to reason under uncertainty using probability.

#### Key Points :

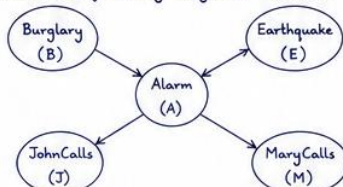
- Nodes represent random variables.
- Directed edges represent causal (dependency) relations.
- Each node has a Conditional Probability Table (CPT) that quantifies the effect of its parents.
- Joint probability is computed using the chain rule.

#### Joint Probability Formula :

$$P(X_1, X_2, \dots, X_n) = \prod_{i=1}^n P(X_i \mid \text{Parents}(X_i))$$

#### Example : (Burglary Alarm Network)

Consider the following Bayesian Network.



B, E are parent nodes of A. A is parent of J and M.

#### Applications :

- Medical diagnosis
- Fault detection
- Decision making under uncertainty
- Planning and prediction

#### Advantages :

- Handles uncertainty
- Compact representation
- Efficient inference
- Causal reasoning

#### Conditional Probability Tables (CPTs) :

| P(B) |       | P(E) |       | P(A   B, E) |   |        |        |
|------|-------|------|-------|-------------|---|--------|--------|
| B    | P(B)  | E    | P(E)  | B           | E | P(A=T) | P(A=F) |
| T    | 0.001 | T    | 0.002 | T           | T | 0.95   | 0.05   |
| F    | 0.999 | F    | 0.998 | T           | F | 0.94   | 0.06   |
|      |       |      |       | F           | T | 0.29   | 0.71   |
|      |       |      |       | F           | F | 0.001  | 0.999  |

| P(J   A) |        |        |
|----------|--------|--------|
| A        | P(J=T) | P(J=F) |
| T        | 0.90   | 0.10   |
| F        | 0.05   | 0.95   |

| P(M   A) |        |        |
|----------|--------|--------|
| A        | P(M=T) | P(M=F) |
| T        | 0.70   | 0.30   |
| F        | 0.01   | 0.99   |

#### Inference Example :

Find the probability that it is Burglary given that John calls.

That is,  $P(B = T \mid J = T) = ?$

Using Bayes' theorem :

$$P(B|J) = \frac{P(J|B) P(B)}{P(J)}$$

We compute :

$$\begin{aligned}
 P(J) &= P(J|B) P(B) + P(J|\neg B) P(\neg B) \\
 &= (0.90)(0.001) + (0.05)(0.999) = 0.00090 + 0.04995 \\
 &= 0.05085
 \end{aligned}$$

$$P(B|J) = \frac{(0.90)(0.001)}{0.05085} = \frac{0.00090}{0.05085} \approx 0.0177$$

Therefore,  $P(\text{Burglary} \mid \text{John calls}) \approx 0.0177 (1.77\%)$



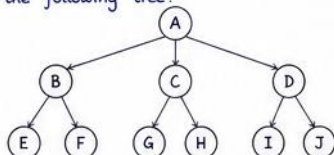
## 1. DIFFERENCE BETWEEN BFS AND DFS

### Definitions :

- Breadth First Search (BFS) explores the nodes level by level from the start node using a queue (FIFO).
- Depth First Search (DFS) explores as far as possible along each branch before backtracking using a stack (LIFO) or recursion.

### Example :

Consider the following tree.



BFS Traversal (Level order) : A, B, C, D, E, F, G, H, I, J

DFS Traversal (Left to right) : A, B, E, F, C, G, H, D, I, J

| Feature          | BFS                    | DFS                                          |
|------------------|------------------------|----------------------------------------------|
| Strategy         | Level by level         | Go deep along a branch                       |
| Data Structure   | Queue (FIFO)           | Stack (LIFO) / Recursion                     |
| Time Complexity  | $O(b^d)$               | $O(b^m)$                                     |
| Space Complexity | $O(b^d)$               | $O(bm)$                                      |
| Completeness     | Yes (if b is finite)   | No (fails in infinite depth)                 |
| Optimality       | Yes (unit cost)        | No                                           |
| Best Used For    | Find shallow solutions | Find solutions in deep space with low memory |

where,  $b$  = branching factor,  $d$  = depth of shallowest goal,  
 $m$  = maximum depth of the tree.

## 2. UNINFORMED VS INFORMED SEARCH

### Definitions :

- Uninformed (Blind) Search : Uses only the problem definition (initial state, actions, goal test, path cost). No extra knowledge about the goal location.
- Informed (Heuristic) Search : Uses additional domain knowledge (heuristic function  $h(n)$ ) to guide the search towards the goal. More efficient.

### Comparison Table :

| Aspect             | Uninformed Search                           | Informed Search                           |
|--------------------|---------------------------------------------|-------------------------------------------|
| Knowledge Used     | Only problem information                    | Problem information + heuristic knowledge |
| Guidance           | No guidance                                 | Guided towards goal                       |
| Examples           | BFS, DFS, Uniform Cost, Iterative Deepening | Greedy Best-First, A*, Hill Climbing      |
| Efficiency         | Generally less efficient                    | Generally more efficient                  |
| Guarantee          | Some are complete & optimal                 | Depends on heuristic (may not be optimal) |
| Heuristic Function | Not used ( $h(n) = 0$ )                     | Used ( $h(n) > 0$ )                       |

Note : A\* search is an informed search that is both complete and optimal if the heuristic is admissible and consistent.

## 3. HILL CLIMBING ALGORITHM

### Definition :

Hill Climbing is a local search algorithm that starts with an initial state and repeatedly moves to the neighboring state with a better heuristic value. It stops when no better neighbor is found.

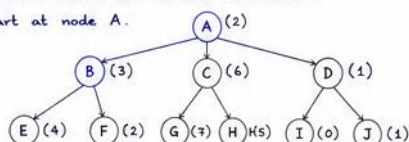
### Algorithm :

1. Choose an initial state  $n$ .
2. Evaluate all neighbors of  $n$  using heuristic function  $h(n)$ .
3. Select the neighbor  $n'$  with the best value (highest  $h(n)$  or lowest  $h(n)$  depending on the problem).
4. If  $h(n')$  is better than  $h(n)$ , move to  $n'$  and repeat from step 2.
5. If no neighbor is better, stop and return current state as local optimum.

### Example : (Maximization problem)

Heuristic values are written in brackets.

Start at node A.



### Steps :

A(2) → neighbors {B(3), C(6), D(1)}  
 Choose C(6)  
 From C → neighbors {G(7), H(5)}  
 Choose G(7)  
 G has no better neighbor.  
 Hence, stop at G(7) (local optimum)

### Characteristics / Issues :

- Simple and fast.
- May get stuck in local optimum.
- Not complete (may fail to find goal).
- Not optimal.
- Variants : Steepest-Ascent, Stochastic Hill Climbing, Simulated Annealing.

## 4. PROPERTIES OF SEARCH ALGORITHMS

The main properties used to evaluate search algorithms are Completeness and Optimality.

### 1. Completeness :

An algorithm is complete if it is guaranteed to find a solution if one exists.

- BFS, Uniform Cost, Iterative Deepening → Complete (if  $b$  is finite).
- DFS, Depth-Limited Search → Not complete (may fail in infinite depth).
- Hill Climbing, Greedy Best-First → Not complete (can get stuck in local optimum).

### 2. Optimality :

An algorithm is optimal if it always finds the least-cost (best) solution.

- BFS → Optimal if all step costs are equal.
- Uniform Cost Search → Optimal.
- A\* → Optimal if heuristic is admissible and consistent.
- DFS, Hill Climbing, Greedy Best-First → Not optimal.

### Summary Table of Properties

| Algorithm            | Complete ? | Optimal ?       | Remarks                              |
|----------------------|------------|-----------------|--------------------------------------|
| Breadth First Search | Yes        | Yes (unit cost) | Explores level by level              |
| Depth First Search   | No         | No              | May go infinite deep                 |
| Uniform Cost Search  | Yes        | Yes             | Expands least cost first             |
| Iterative Deepening  | Yes        | Yes (unit cost) | Like BFS but less memory             |
| Hill Climbing        | No         | No              | Gets stuck in local optimum          |
| Greedy Best-First    | No         | No              | Fast but not optimal                 |
| A* Search            | Yes*       | Yes*            | If $h(n)$ is admissible & consistent |

\* Depends on the heuristic used.

### Key Takeaway :

BFS and Uniform Cost are reliable but may use more memory. DFS uses less memory but is not reliable. Informed searches like A\* use domain knowledge to achieve better performance.



## 1. DIFFERENCE BETWEEN PROPOSITIONAL AND PREDICATE LOGIC

### Definition :

- Propositional Logic (PL) deals with simple propositions that are either True or False. It does not consider the internal structure of objects.
- Predicate Logic (First Order Logic - FOL) extends PL by using predicates, variables, functions and quantifiers to reason about objects and their relations.

### Comparison Table :

| Feature                  | Propositional Logic (PL)                   | Predicate Logic (FOL)                                                                     |
|--------------------------|--------------------------------------------|-------------------------------------------------------------------------------------------|
| Basic Unit               | Proposition (e.g. P, Q)                    | Predicate with arguments (e.g. P(x), Loves(x, y))                                         |
| Structure                | Has no internal structure                  | Has internal structure (objects, relations)                                               |
| Variables                | No variables                               | Uses variables (x, y, z, ...)                                                             |
| Quantifiers              | No quantifiers                             | Uses $\forall$ (for all), $\exists$ (there exists)                                        |
| Expressiveness           | Less expressive                            | More expressive                                                                           |
| Knowledge Representation | Facts about propositions                   | Facts about objects and their relations                                                   |
| Example                  | P: It is raining.<br>Q: The ground is wet. | Rains(x): x is raining.<br>Wet(y): y is wet.<br>$\forall x (Rains(x) \rightarrow Wet(x))$ |
| Reasoning                | Simpler, decidable                         | More complex, semi-decidable                                                              |
| Applications             | Digital circuits, simple AI problems       | Natural language understanding, knowledge bases, expert systems                           |

### Examples :

#### (i) Propositional Logic

- P: It is raining.
- Q: The ground is wet.
- Rule:  $P \rightarrow Q$
- Fact: P

Conclusion: Q (by Modus Ponens)

#### (ii) Predicate Logic (FOL)

- Rains(x): x is raining.
- Wet(x): x is wet.
- Rule:  $\forall x (Rains(x) \rightarrow Wet(x))$
- Fact: Rains(Chennai)

Conclusion: Wet(Chennai)

(Universal Instantiation + Modus Ponens)

### Key Takeaway :

PL reasons about facts.

FOL reasons about objects, their properties and relations.

## 2. INFERENCE IN FIRST ORDER LOGIC (FOL)

### Definition :

Inference in FOL is the process of deriving new sentences (conclusions) from a knowledge base (set of FOL sentences) using rules of inference that are sound and valid.

### Common Rules of Inference in FOL :

- Universal Instantiation (UI): From  $\forall x P(x)$ , infer  $P(c)$  for any constant  $c$ .
- Existential Instantiation (EI): From  $\exists x P(x)$ , infer  $P(c)$  for a new constant  $c$ .
- Universal Generalization (UG): From  $P(c)$  ( $c$  arbitrary), infer  $\forall x P(x)$ .
- Existential Generalization (EG): From  $P(c)$ , infer  $\exists x P(x)$ .
- Modus Ponens (MP): From  $P$  and  $(P \rightarrow Q)$ , infer  $Q$ .
- And Elimination ( $\wedge E$ ): From  $P \wedge Q$ , infer  $P$  (or  $Q$ ).
- And Introduction ( $\wedge I$ ): From  $P$  and  $Q$ , infer  $P \wedge Q$ .

### Example :

#### Knowledge Base (KB):

- $\forall x (\text{Human}(x) \rightarrow \text{Mortal}(x))$
- $\text{Human}(\text{Socrates})$

Query:  $\text{Mortal}(\text{Socrates})$

#### Proof (using inference rules):

| Step | Sentence                                                                   | Rule                |
|------|----------------------------------------------------------------------------|---------------------|
| 1    | $\forall x (\text{Human}(x) \rightarrow \text{Mortal}(x))$                 | Premise             |
| 2    | $\text{Human}(\text{Socrates})$                                            | Premise             |
| 3    | $\text{Human}(\text{Socrates}) \rightarrow \text{Mortal}(\text{Socrates})$ | UI from (1)         |
| 4    | $\text{Mortal}(\text{Socrates})$                                           | MP from (2) and (3) |

Conclusion:  $\text{Mortal}(\text{Socrates})$  is inferred from the KB.

## 3. BAYESIAN REASONING

### Definition :

Bayesian reasoning is a method of reasoning under uncertainty based on Bayes' Theorem. It allows us to update the probability of a hypothesis given new evidence.

#### Bayes' Theorem :

$$P(H|E) = \frac{P(E|H) P(H)}{P(E)}$$

where,

- $P(H|E)$  : Posterior probability (probability of H given E)
- $P(E|H)$  : Likelihood (probability of E given H)
- $P(H)$  : Prior probability (initial belief in H)
- $P(E)$  : Evidence probability (total probability of E)

### Example : Medical Diagnosis

A disease D affects 1% of a population.

A test T is 95% accurate:

- $P(T+|D) = 0.95$  (test positive given disease)
- $P(T+|\neg D) = 0.05$  (test positive given no disease)

If a person tests positive (T+), what is the probability that he actually has the disease (D)?

#### Solution :

$$P(D) = 0.01, \quad P(\neg D) = 0.99$$

$$P(T+|D) = 0.95, \quad P(T+|\neg D) = 0.05$$

Using Bayes' Theorem,

$$\begin{aligned} P(D|T+) &= \frac{P(T+|D) P(D)}{P(T+|D) P(D) + P(T+|\neg D) P(\neg D)} \\ &= \frac{0.95 \times 0.01}{(0.95 \times 0.01) + (0.05 \times 0.99)} = \frac{0.0095}{0.0095 + 0.0495} = 0.161 \end{aligned}$$

Therefore,  $P(D|T+) \approx 0.161$  or 16.1%

## 1. EXPLAIN FIRST ORDER LOGIC (FOL)

### Definition :

First Order Logic (FOL) is a formal logic that allows reasoning about objects, their properties and relations. It extends propositional logic by using predicates, functions, variables and quantifiers.

### Key Components :

- Constants : Specific objects (e.g., John, Socrates)
- Variables : Range over objects (e.g.,  $x, y$ )
- Predicates : Properties or relations (e.g.,  $\text{Human}(x)$ ,  $\text{Loves}(x, y)$ )
- Functions : Map objects to objects (e.g.,  $\text{Father}(x)$ )
- Quantifiers :  $\forall$  (for all),  $\exists$  (there exists)
- Logical Connectives :  $\neg$ ,  $\wedge$ ,  $\vee$ ,  $\rightarrow$ ,  $\leftrightarrow$

### Syntax (well-formed formula) :

FOL formula ::= Predicate |  $\neg$ FOL | FOL  $\wedge$  FOL | FOL  $\vee$  FOL | FOL  $\rightarrow$  FOL | FOL  $\leftrightarrow$  FOL |  $\forall x$  FOL |  $\exists x$  FOL

### Examples :

1.  $\forall x (\text{Human}(x) \rightarrow \text{Mortal}(x))$   
(For all  $x$ , if  $x$  is human then  $x$  is mortal.)
2.  $\text{Human}(\text{Socrates})$   
(Socrates is a human.)
3.  $\text{Mortal}(\text{Socrates})$   
(Socrates is mortal.)
4.  $\exists x (\text{Student}(x) \wedge \text{Loves}(x, \text{AI}))$   
(There exists a student who loves AI.)

### Knowledge Base Example :

1.  $\forall x (\text{Human}(x) \rightarrow \text{Mortal}(x))$
2.  $\text{Human}(\text{Socrates})$

Query :  $\text{Mortal}(\text{Socrates})$

### Explanation :

From the KB, anyone who is human is mortal.  
Socrates is human. Therefore, Socrates is mortal.

## 2. EXPLAIN RESOLUTION / THEOREM PROVING

### Definition :

Resolution is a rule of inference used in theorem proving. It works by deriving new clauses from existing clauses until either an empty clause ( $\square$ ) is derived (proof succeeds) or no new information can be derived (proof fails).

### Key Points :

- KB is converted to Conjunctive Normal Form (CNF).
- Negate the query and add it to KB.
- Repeatedly apply resolution rule to derive new clauses.
- If empty clause ( $\square$ ) is obtained  $\Rightarrow$  query is entailed (proved).
- Otherwise, query is not entailed.

### Resolution Rule :

$$\frac{(A \vee P), (\neg P \vee B)}{\therefore (A \vee B)}$$

where  $P$  is any literal.

### Example :

- KB : 1.  $\forall x (\text{Human}(x) \rightarrow \text{Mortal}(x))$   
2.  $\text{Human}(\text{Socrates})$

Query :  $\text{Mortal}(\text{Socrates})$

### Step 1: Convert KB and $\neg$ Query to CNF

- $\forall x (\text{Human}(x) \rightarrow \text{Mortal}(x)) \equiv \forall x (\neg \text{Human}(x) \vee \text{Mortal}(x))$   
After removing  $\forall$  :  $(\neg \text{Human}(x) \vee \text{Mortal}(x))$
- $\text{Human}(\text{Socrates})$
- $\neg \text{Mortal}(\text{Socrates})$  (negation of query)

### Step 2: List of clauses

- C1 :  $(\neg \text{Human}(x) \vee \text{Mortal}(x))$   
C2 :  $(\text{Human}(\text{Socrates}))$   
C3 :  $(\neg \text{Mortal}(\text{Socrates}))$

### Step 3: Apply resolution

| Resolve                                      | From Clauses                                                                            | Result                             | Explanation                                 |
|----------------------------------------------|-----------------------------------------------------------------------------------------|------------------------------------|---------------------------------------------|
| C1 and C2                                    | $(\neg \text{Human}(x) \vee \text{Mortal}(x))$<br>and $(\text{Human}(\text{Socrates}))$ | $(\text{Mortal}(\text{Socrates}))$ | Unify $x$ with Socrates, resolve on Human   |
| $(\text{Mortal}(\text{Socrates}))$<br>and C3 | $(\text{Mortal}(\text{Socrates}))$<br>and $(\neg \text{Mortal}(\text{Socrates}))$       | $\square$ (empty clause)           | Resolve on $\text{Mortal}(\text{Socrates})$ |

Since empty clause is derived, the query  $\text{Mortal}(\text{Socrates})$  is proved.  
Hence, the KB entails the query.

## 3. EXPLAIN BAYESIAN NETWORK

### Definition :

A Bayesian Network (BN) is a graphical model that represents a set of random variables and their conditional dependencies using a Directed Acyclic Graph (DAG). It allows reasoning under uncertainty using probability.

### Key Points :

- Nodes represent random variables.
- Directed edges represent direct (causal) dependencies.
- Each node has a Conditional Probability Table (CPT).
- The joint probability is the product of all CPTs.

### Joint Probability Formula :

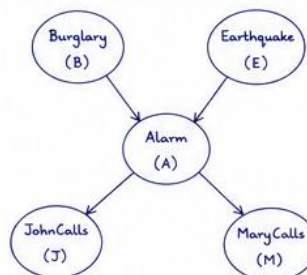
$$P(X_1, X_2, \dots, X_n) = \prod_{i=1}^n P(X_i | \text{Parents}(X_i))$$

where  $\text{Parents}(X_i)$  are the direct predecessors of  $X_i$ .

### Inference Example :

Find  $P(\text{Burglary} = T | \text{JohnCalls} = T, \text{MaryCalls} = T)$   
This can be computed using Bayes' theorem and the CPTs via enumeration or variable elimination.

### Example : Burglary Alarm Network



### P(B)

| B | P(B)  |
|---|-------|
| T | 0.001 |
| F | 0.999 |

### P(E)

| E | P(E)  |
|---|-------|
| T | 0.002 |
| F | 0.998 |

### P(A | B, E)

| B | E | P(A=T) | P(A=F) |
|---|---|--------|--------|
| T | T | 0.95   | 0.05   |
| T | F | 0.94   | 0.06   |
| F | T | 0.29   | 0.71   |
| F | F | 0.001  | 0.999  |

### P(J | A)

| A | P(J=T) | P(J=F) |
|---|--------|--------|
| T | 0.90   | 0.10   |
| F | 0.05   | 0.95   |

### P(M | A)

| A | P(M=T) | P(M=F) |
|---|--------|--------|
| T | 0.70   | 0.30   |
| F | 0.01   | 0.99   |

BNs are widely used in diagnosis, prediction, decision making, and other real-world AI applications.



## 1. EXPLAIN A\* ALGORITHM WITH EXAMPLE

### Definition:

A\* (A-star) algorithm is an informed search algorithm that finds the optimal path from start node to goal node using the evaluation function.

$$f(n) = g(n) + h(n)$$

where,

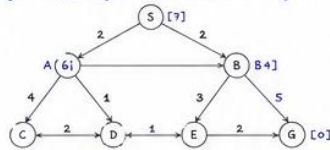
- $g(n)$  = cost from start node to  $n$
- $h(n)$  = heuristic estimate from  $n$  to goal

### Steps of A\* Algorithm:

- Put the start node in OPEN list and set  $g(\text{start}) = 0$ .
- For the node  $n$  in OPEN with smallest  $f(n)$ ,
  - Remove  $n$  from OPEN and put in CLOSED.
  - If  $n$  is the goal, return the path.
  - Generate all successors of  $n$ .
  - For each successor  $s$ , compute  $g(s)$  and  $f(s)$ .
  - Add  $s$  to OPEN if it is not in OPEN or CLOSED or if a cheaper path is found.
- Repeat Step 2 until goal is found or OPEN is empty.

### Example:

Consider the graph below. Heuristic  $h(n)$  to goal  $G$  is given in []. Edge costs are shown on the edges.



OPEN = {S},  $g(S)=0$ ,  $f(S)=0+7=7$

### A\* Search Trace:

| Step | Node Expanded | OPEN List (Node: g, h, f) |
|------|---------------|---------------------------|
| 1    | S             | A:2,6,8 B:2,4,6           |
| 2    | B             | A:2,6,8 E:5,3,8 G:7,0,7   |
| 3    | G             | (Goal found)              |

### Optimal Path:

$S \rightarrow B \rightarrow G$   
Total Cost =  $2 + 5 = 7$

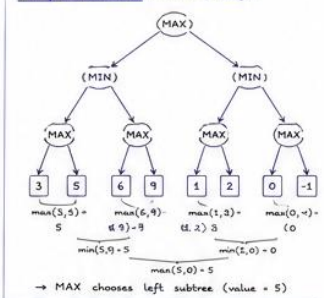
Note: A\* is complete and optimal if  $h(n)$  is admissible (never over-estimates) and consistent.

## 2. EXPLAIN MINIMAX + ALPHA-BETA PRUNING

### Minimax Algorithm:

- Used in two-player, zero-sum games.
- MAX player tries to maximize utility.
- MIN player tries to minimize utility.
- The algorithm explores the game tree up to terminal states and assigns values using:
  - MAX nodes  $\rightarrow$  maximum of children
  - MIN nodes  $\rightarrow$  minimum of children
- The root value gives the best achievable value for MAX.

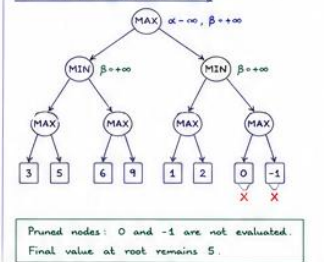
### Example Game Tree (without pruning):



### Alpha-Beta Pruning:

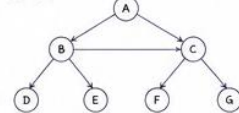
- Optimization of minimax.
- Maintains two values:
  - $\alpha$  (alpha)  $\rightarrow$  best value for MAX so far
  - $\beta$  (beta)  $\rightarrow$  best value for MIN so far
- If at any node,  $\alpha \geq \beta$ , remaining children need not be explored (pruned).

### Same Tree with Alpha-Beta Pruning



## 3. SOLVE SEARCH PROBLEM (GRAPH)

Problem: Find the shortest path from A to G.  
Use BFS.



### BFS Solution:

Queue (FIFO) is used.

### Traversal Steps:

- Start at A. Queue = [A]
- Visit A  $\rightarrow$  add B, C. Queue = [B, C]
- Visit B  $\rightarrow$  add D, E. Queue = [C, D, E]
- Visit C  $\rightarrow$  add F, G. Queue = [D, E, F, G]
- Visit D  $\rightarrow$  Queue = [E, F, G]
- Visit E  $\rightarrow$  Queue = [F, G]
- Visit F  $\rightarrow$  Queue = [G]
- Visit G  $\rightarrow$  Goal found.

Shortest path (one of the shortest paths):  
 $A \rightarrow C \rightarrow G$  (Cost = 2)

### DFS Solution (for comparison):

Use Stack (LIFO).

### Traversal Steps:

- Start at A. Stack = [A]
- Pop A  $\rightarrow$  push C, B. Stack = [C, B]
- Pop B  $\rightarrow$  push E, D. Stack = [C, E, D]
- Pop D  $\rightarrow$  Stack = [C, E]
- Pop E  $\rightarrow$  Stack = [C]
- Pop C  $\rightarrow$  push G, F. Stack = [G, F]
- Pop F  $\rightarrow$  Stack = [G]
- Pop G  $\rightarrow$  Goal found.

Path found by DFS:  
 $A \rightarrow B \rightarrow E \rightarrow C \rightarrow G$   
(Not the shortest path)

## 4. EXPLAIN FIRST ORDER LOGIC (FOL) + INFERENCE

### First Order Logic (FOL)

#### Definition:

FOL is a formal language with variables, predicates, functions and quantifiers to represent objects, properties and relations.

#### Components:

- Constants: John, Socrates
- Variables:  $x, y, z$
- Predicates: Human( $x$ ), Loves( $x, y$ )
- Functions: Father( $x$ )
- Quantifiers:  $\forall$  (for all),  $\exists$  (there exists)

#### Example Sentences:

- $\forall x (\text{Human}(x) \rightarrow \text{Mortal}(x))$   
(All humans are mortal.)
- Human(Socrates)  
(Socrates is a human.)
- $\exists x (\text{Student}(x) \wedge \text{Loves}(x, \text{AI}))$   
(There exists a student who loves AI.)

### Inference in FOL

Inference derives new sentences from KB using logical rules.

#### Common Rules:

- Universal Instantiation (UI):  
 $\forall x P(x) \rightarrow P(c)$
- Existential Instantiation (EI):  
 $\exists x P(x) \rightarrow P(k)$  ( $k$  is a new constant)
- Universal Generalization (UG):  
 $P(x) \rightarrow \forall x P(x)$
- Existential Generalization (EG):  
 $P(c) \rightarrow \exists x P(x)$

#### Example Inference:

KB:  $\forall x (\text{Human}(x) \rightarrow \text{Mortal}(x))$ , Human(Socrates)  
To prove: Mortal(Socrates)  
1.  $\forall x (\text{Human}(x) \rightarrow \text{Mortal}(x))$  (Premise)  
2. Human(Socrates) (Premise)  
3.  $\exists x (\text{Student}(x) \wedge \text{Loves}(x, \text{AI}))$  (UI from 1)  
4. Mortal(Socrates) (Modus Ponens from 2,3)  
 $\therefore$  Mortal(Socrates) is inferred.

## 5. EXPLAIN BAYESIAN NETWORKS

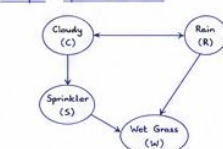
### Definition:

A Bayesian Network (BN) is a directed acyclic graph (DAG) where nodes represent random variables and edges represent conditional dependencies. It allows reasoning under uncertainty using probability.

### Key Points:

- Each node has a Conditional Probability Table (CPT).
- Joint probability is the product of all CPTs.
- BN compactly represents the joint distribution.

### Example: Sprinkler Network



### CPTs:

| P(C) |      | P(S C) |      | P(R C) |      |
|------|------|--------|------|--------|------|
| C    | P(C) | C      | S    | C      | R    |
| T    | 0.5  | T      | 0.10 | T      | 0.80 |
| F    | 0.5  | T      | 0.90 | T      | 0.20 |
|      |      | F      | 0.50 | F      | 0.20 |
|      |      | F      | 0.50 | F      | 0.80 |

| P(W S, R) |   |   |      |
|-----------|---|---|------|
| S         | R | W | P(W) |
| T         | T | F | 0.99 |
| T         | T | F | 0.01 |
| T         | F | F | 0.90 |
| T         | F | F | 0.10 |
| F         | T | F | 0.90 |
| F         | T | F | 0.01 |
| F         | F | F | 0.99 |

Joint Probability:  
 $P(C, S, R, W) = P(C) P(S|C) P(R|C) P(W|S, R)$

## 6. EXPLAIN DECISION TREE / NEURAL NETWORK

### A. Decision Tree

#### Definition:

A decision tree is a flowchart-like structure where internal nodes test attributes, branches represent outcomes and leaves represent decisions or classes.

#### Example: Play Tennis



#### Advantages:

- Easy to understand
- Handles numerical & categorical data

#### Disadvantage:

- Proned to overfitting

### B. Neural Network

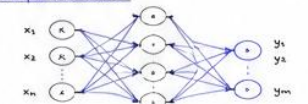
#### Definition:

A neural network is a computational model inspired by the human brain. It consists of layers of interconnected nodes (neurons) that learn patterns from data.

#### Structure:

- Input Layer  $\rightarrow$  receives inputs
- Hidden Layer(s)  $\rightarrow$  processes information
- Output Layer  $\rightarrow$  produces output

#### Example Architecture:



#### Advantages:

- Can learn complex patterns
- Good for large & noisy data

#### Disadvantages:

- Needs more data & computations

## 7. COMPARE SEARCH ALGORITHMS

| Algorithm               | Type       | Data Structure           | Complete?              | Optimal? | Time Complexity  | Space Complexity | Best Used For        |
|-------------------------|------------|--------------------------|------------------------|----------|------------------|------------------|----------------------|
| BFS                     | Uninformed | Queue (FIFO)             | Yes*                   | Yes*     | $O(b^*)$         | $O(b^*)$         | Shallow solutions    |
| DFS                     | Uninformed | Stack (LIFO) / Recursion | No                     | No       | $O(b^m)$         | $O(b^m)$         | Memory limited       |
| Uniform Cost            | Uninformed | Priority Queue           | Yes                    | Yes      | $O(b^{1+ C /e})$ | $O(b^{1+ C /e})$ | Varying costs        |
| Depth Limited           | Uninformed | Stack                    | Yes (if $d \leq d_l$ ) | No       | $O(b^d)$         | $O(b^d)$         | Avoid infinite depth |
| Iterative Deepening DFS | Uninformed | Stack                    | Yes                    | Yes*     | $O(b^d)$         | $O(b^d)$         | Unknown depth        |
| Greedy Best-First       | Informed   | Priority Queue           | No                     | No       | $O(b^m)$         | $O(b^m)$         | Fast (approx.)       |
| A*                      | Informed   | Priority Queue           | Yes*                   | Yes*     | Exponential      | Exponential      | Optimal solution     |

Where,  $b$  = branching factor,  $d$  = depth of shallowest goal,  $m$  = maximum depth,  $C^*$  = optimal cost,  $e$  = minimum step cost, \* = under suitable conditions.

## 8. SUMMARY / KEY TAKEAWAYS

- A\* uses  $f(n)=g(n)+h(n)$  and is optimal with admissible heuristic.
- Minimax finds the best move for MAX; Alpha-Beta pruning reduces the number of nodes evaluated.
- BFS is complete and optimal for unit costs; DFS uses less memory but is not complete.
- FOL extends propositional logic with objects, relations and quantifiers. Inference uses rules like UI, EI, Modus Ponens.
- Bayesian Networks represent probabilistic relationships and use CPTs for reasoning.
- Decision Trees are simple and interpretable; Neural Networks learn complex patterns.
- Choice of search algorithm depends on problem requirements like completeness, optimality, time and memory.

Understanding these concepts helps in building intelligent systems that can reason, learn and make decisions.



### 1. Define Artificial Intelligence (AI)

AI is the field of computer science that aims to create intelligent agents, which are systems that can perceive their environment, reason, learn and act to achieve goals autonomously.

### 2. What is an Intelligent Agent?

An intelligent agent is an entity that perceives its environment through sensors and acts upon that environment through actuators to achieve goals.



### 3. Define State Space

A state space is a set of all possible states of a problem and the transitions between them. It is represented as a graph where nodes are states and edges are actions.

Example:

In 8-puzzle, each configuration of tiles is a state.

### 4. What is a Heuristic Function?

A heuristic function  $h(n)$  is a function that estimates the cost from node  $n$  to the goal.

- It guides informed search algorithms.
- A good heuristic never overestimates the actual cost (admissible heuristic).

Example:

In 8-puzzle,  $h(n)$  can be the number of misplaced tiles.

### 5. Define Uninformed Search

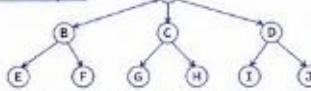
Uninformed search (blind search) uses no problem-specific knowledge. It explores the state space systematically.

Examples: BFS, DFS, Uniform Cost Search.

### 6. What is BFS / DFS?

- BFS (Breadth-First Search) explores the shallowest nodes first using a queue (FIFO). It is complete and optimal (for equal step cost).
- DFS (Depth-First Search) explores as deep as possible along a branch before backtracking using a stack (LIFO). It uses less memory but may not be complete.

BFS Example:



BFS Order: A, B, C, D, E, F, G, H, I, J

DFS Order (Left to right): A, B, E, F, C, G, H, D, I, J

### 7. Define A\* Algorithm

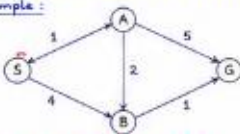
A\* (A-star) is an informed search algorithm that uses evaluation function  $f(n) = g(n) + h(n)$

where,  $g(n)$  = cost from start to  $n$

$h(n)$  = heuristic cost from  $n$  to goal

It finds the optimal path if  $h(n)$  is admissible and consistent.

Example:



Heuristic  $h(n)$ :

$h(S)=5, h(A)=4,$

$h(B)=1, h(G)=0$

A\* finds path

$S \rightarrow B \rightarrow G$  with cost = 5 (optimal)

### 8. What is Hill Climbing?

Hill Climbing is a local search algorithm that starts from an initial state and repeatedly moves to the neighbor with better heuristic value. It stops when no better neighbor exists.

- Simple and fast
- Can get stuck in local maxima

Example: Finding highest peak in a landscape.



### 9. Define Minimax Algorithm

Minimax is a decision rule used in two-player, zero-sum games. MAX player tries to maximize the utility, while MIN player tries to minimize it.

- MAX selects the move with maximum minimum value.
- MIN selects the move with minimum maximum value.

### 10. What is Alpha-Beta Pruning?

Alpha-Beta pruning is an optimization technique for minimax. It eliminates branches that will not affect the final decision, thus reducing the number of nodes evaluated.

- Alpha ( $\alpha$ ): best value already found for MAX
- Beta ( $\beta$ ): best value already found for MIN

If  $\alpha \geq \beta$ , the remaining nodes are pruned.



### 11. Define Propositional Logic

Propositional Logic (PL) is a formal system where knowledge is represented as propositions that can be either True or False using logical connectives ( $\neg, \wedge, \vee, \rightarrow, \leftrightarrow$ ).

Example: P: It is raining. Q: The ground is wet.

Rule:  $P \rightarrow Q$

### 12. What is Predicate Logic (FOL)?

Predicate Logic (First Order Logic) extends propositional logic by using predicates, variables, functions and quantifiers to express knowledge about objects and their relations.

- Predicates:  $\text{Human}(x), \text{Loves}(x, y)$
- Quantifiers:  $\forall$  (for all),  $\exists$  (there exists)

Example:  $\forall x (\text{Human}(x) \rightarrow \text{Mortal}(x))$

$\exists x (\text{Student}(x) \wedge \text{Loves}(x, \text{AI}))$

### 13. Define Knowledge Base

A Knowledge Base (KB) is a collection of sentences in a formal language that represents the knowledge of a domain.

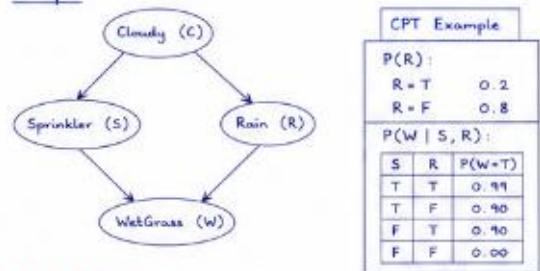
It is used by an inference engine to derive new facts.

Example:  $\text{KB} = \{\text{Human}(\text{Socrates}), \text{Human}(x) \rightarrow \text{Mortal}(x)\}$

### 14. What is Bayesian Network?

A Bayesian Network (BN) is a directed acyclic graph (DAG) where nodes represent random variables and edges represent conditional dependencies. Each node has a Conditional Probability Table (CPT).

Example:



### 15. Define Machine Learning

Machine Learning (ML) is a subfield of AI that enables systems to learn patterns from data and improve performance on tasks without being explicitly programmed.

Types: Supervised, Unsupervised, Reinforcement Learning.

### 16. What is Supervised Learning?

Supervised Learning uses labeled training data (input-output pairs) to learn a mapping function that can predict outputs for new inputs.

Examples: Classification, Regression.

Example: Predicting house prices using features like area, bedrooms, location.

### 17. What is Neural Network?

A Neural Network (NN) is a computational model inspired by the human brain. It consists of interconnected layers of neurons that learn patterns from data.

Structure:

Input Layer  $\rightarrow$  Hidden Layer(s)  $\rightarrow$  Output Layer



### 18. Define Expert System

An Expert System is an AI system that uses a knowledge base and inference engine to solve problems that normally require human expertise.

Example: MYCIN (medical diagnosis), DENDRAL (chemistry).

### 19. What is Natural Language Processing (NLP)?

Natural Language Processing (NLP) is a field of AI that enables computers to understand, interpret and generate human language.

Applications: Machine translation, Sentiment analysis, Chatbots, Speech recognition.